Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Actividad Autónoma — Sesión 8

Pruebas de dependencias entre Categorías y Productos

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	8 · 1 de octubre de 2026
Modalidad	Individual, sin acompañamiento docente	Plazo	Hasta el 7 de octubre de 2026, 23:59
Entrega	Aula virtual + repositorio Git	Puntaje	20 puntos (evaluación de sesión)
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. De dónde partimos

En la actividad práctica (Reto 01) agregaste a PharmaSoft el módulo Productos: el jefe de almacén ya elige la categoría de una lista, la SPA no le ofrece categorías inactivas y los errores del backend se muestran con un mensaje claro. Sin embargo, que la interfaz se vea bien no garantiza que las dependencias estén protegidas. Un usuario con Postman, dos pestañas abiertas al mismo tiempo o un cambio de estado en la categoría pueden producir datos que la SPA nunca permitiría.

Ahora debes comportarte como el responsable de calidad del proyecto: probar sistemáticamente qué ocurre con cada alta, cambio y baja, válida o inválida, y documentar lo que encuentres.

Reto 02

Diseña y ejecuta una matriz de pruebas de altas, cambios y bajas válidas e inválidas entre Categorías y Productos, contrasta el resultado esperado con el obtenido en la SPA y en la API, y reporta los hallazgos con una corrección propuesta para cada uno.

2. Actividad autónoma

Esta actividad extiende el Reto 01 sobre el mismo repositorio y el mismo backend. No se trata de escribir mucho código nuevo, sino de **verificar** con método: cada caso tiene una precondición, unos datos, un resultado esperado y un resultado obtenido. Donde la SPA impide una operación, la probarás también con Postman,

porque una regla que solo existe en la interfaz no protege los datos. Si te bloqueas, revisa la sección 4, «Dudas frecuentes».

Campo	Contenido
Título	Matriz de pruebas e informe de hallazgos: dependencias entre Categorías y Productos
Objetivo	Diseña y ejecuta una matriz de pruebas de altas, cambios y bajas válidas e inválidas entre Categorías y Productos, contrasta el resultado esperado con el obtenido en la SPA y en la API, y justifica una corrección para cada hallazgo indicando la capa donde debe implementarse.
Tiempo estimado	4 a 5 horas
Materiales	Guía práctica de la sesión 8, tu SPA PharmaSoft, PharmaBackend (`master`) en ejecución, Swagger UI y Postman.

Preparación

- Parte de una base con al menos **tres categorías activas** y **una inactiva**, y con **ocho productos** repartidos entre ellas (al menos uno dado de baja).
- Crea en Postman la colección `PharmaSoft – S8 Dependencias` con la variable `baseUrl = http://localhost:8080/api/v1`. Guarda en ella cada petición que ejecutes.
- Crea la rama `feature/pruebas-dependencias-<apellido>` desde `develop`, después de fusionar tu rama de la práctica.

Ejemplo de petición que la SPA no permite y que debes probar directamente contra la API:

```
POST {{baseUrl}}/productos
Content-Type: application/json

{
  "nombre": "Producto de prueba A-04",
  "precio": 5.50,
  "stock": 10,
  "estado": true,
  "categoriaId": 3        // id de una categoría INACTIVA
}
```

Parte A. Matriz de pruebas (8 puntos)

Construye la matriz en una hoja de cálculo o en Markdown con estas columnas: **ID · Operación · Precondición · Datos · Resultado esperado · Resultado obtenido (SPA y HTTP) · Estado (Pasa / Falla) · Evidencia**. El resultado esperado se deduce de las **reglas del negocio**, no de lo que hace hoy el sistema: si el sistema se comporta distinto, el caso **falla** y es un hallazgo.

La matriz debe tener **al menos 18 casos**: los 14 casos base de la tabla siguiente, completados con su resultado obtenido, y **al menos 4 casos propios** (uno por lo menos de cada grupo: altas, cambios y bajas).

ID	Operación y datos	Resultado esperado según la regla de negocio	Medio
A-01	Registrar producto válido en una categoría activa.	201; aparece en el listado con su categoría.	SPA
A-02	Registrar producto sin elegir categoría.	La SPA no envía la petición. Por API sin `categoriaId`: 400.	SPA + Postman
A-03	Registrar producto con `categoriaId` 999.	404 «Categoria no encontrada…».	Postman

ID	Operación y datos	Resultado esperado según la regla de negocio	Medio
A-04	Registrar producto en una categoría **inactiva**.	La SPA no lo ofrece. Por API debería rechazarse (409).	SPA + Postman
A-05	Registrar producto con un nombre existente en otra capitalización.	409 «Ya existe un producto…».	SPA
A-06	Registrar producto con precio 0 y stock −1.	SPA bloquea; por API 400 con dos campos en `validationErrors`.	SPA + Postman
C-01	Cambiar la categoría de un producto a otra activa.	200; el listado muestra la nueva categoría.	SPA
C-02	Cambiar la categoría de un producto a una **inactiva**.	SPA lo impide; por API debería rechazarse (409).	SPA + Postman
C-03	**Desactivar** una categoría que tiene productos activos.	Decide y justifica la regla esperada (ver B5).	SPA
C-04	Dos pestañas: en la 1 abrir «Nuevo producto» con la categoría X; en la 2 desactivar X; volver a la 1 y registrar.	El producto no debería quedar en una categoría inactiva.	SPA
B-01	Dar de baja un producto activo.	204; la fila pasa a «Inactivo».	SPA
B-02	Dar de baja otra vez el mismo producto.	409 «…ya se encuentra inactivo».	Postman
B-03	Eliminar una categoría **sin** productos.	204; desaparece del listado y del select de Productos.	SPA
B-04	Eliminar una categoría cuyos productos están **todos dados de baja**.	Decide y justifica la regla esperada (ver B5).	SPA

Cómo registrar la evidencia

Para cada caso guarda una captura de la SPA o de Postman donde se vea el **código HTTP** y el **cuerpo de la respuesta**. Cuando la SPA bloquee la operación, captura el mensaje de la interfaz **y** la pestaña Red vacía, para demostrar que no se envió la petición. Nombra las capturas con el ID del caso: `A-04_postman.png`.

Parte B. Informe de hallazgos (7 puntos)

Redacta un informe de 3 a 5 páginas en PDF con las secciones siguientes.

B1. Resumen de la ejecución. Número de casos ejecutados, cuántos pasan y cuántos fallan, por grupo (altas, cambios y bajas).

B2. Ficha por hallazgo. Una ficha por cada caso que falla, con este formato:

Campo	Contenido esperado
ID del hallazgo	H-01, H-02…
Caso relacionado	ID de la matriz (por ejemplo, A-04).
Descripción	Qué ocurrió y por qué contradice la regla de negocio.
Severidad	Alta (datos inconsistentes), media (mala experiencia con datos correctos) o baja (estética).

Campo	Contenido esperado
Evidencia	Captura y petición de Postman.
Causa probable	Archivo y método del backend o del frontend donde falta la validación.
Corrección propuesta	Qué cambiarías, en qué capa y por qué. Puedes incluir un fragmento de código.

B3. Cobertura de reglas. Una tabla con cada regla de dependencia (existencia de la categoría, categoría activa, nombre único, no eliminar categoría con productos, baja lógica) y dónde se valida hoy: solo SPA, solo API, ambas o ninguna.

B4. Recomendación sobre el filtro por categoría. El filtro del listado de productos trabaja sobre la página actual. Explica por qué y propone el cambio completo: el parámetro nuevo del endpoint, el método del repositorio en Spring Data y el cambio en `ProductoService.listar()` del frontend.

B5. Preguntas de análisis. Responde cada una en un párrafo, citando archivos de tu proyecto o de PharmaBackend:

1. ¿Por qué no basta con que la SPA oculte las categorías inactivas? Relaciónalo con tus casos A-04 y C-02.
2. Cuando se desactiva una categoría con productos activos (C-03), ¿qué debería pasar con esos productos: impedir la desactivación, desactivarlos también o permitirlo con una advertencia? Justifica tu decisión pensando en las ventas.
3. En B-04, el backend cuenta también los productos dados de baja al decidir si una categoría se puede eliminar. ¿Estás de acuerdo? ¿Qué pasaría con los reportes de ventas si se permitiera eliminarla?
4. ¿Qué revela el caso C-04 sobre el momento en que se validan las dependencias? ¿Qué capa es la única que puede resolverlo?
5. Si mañana el backend corrige el hallazgo de A-04, ¿qué tendría que cambiar en la SPA? ¿Y qué seguiría igual?

Parte C. Mejora en la SPA: «Ver productos» de una categoría (3 puntos)

Cuando el backend rechaza la eliminación de una categoría porque tiene productos, el usuario no sabe cuáles son. Implementa esta mejora de presentación de información relacionada:

- En el listado de Categorías, agrega un botón **«Ver productos»** que navegue a `/productos?categoriaId=<id>` con `[routerLink]="'/productos'"` y `[queryParams]="{ categoriaId: c.id }"`.
- En `ProductoList`, recibe el parámetro con `readonly categoriaId = input<string>();`. Gracias a `withComponentInputBinding()`, los parámetros de consulta también llegan como inputs.
- En `ngOnInit()`, si llega `categoriaId`, asígnalo a `categoriaFiltro` y deja seleccionada esa opción en el select de filtro (`[value]` o `[selected]`).
- Como el filtro trabaja sobre la página actual, cuando llegue `categoriaId` carga la página con `tamanio` 100 (el máximo que acepta el backend) y muestra una nota: «Mostrando productos de la categoría X en los primeros 100 registros».

Parte D. Versionamiento y entrega (2 puntos)

- Registra al menos tres commits en días distintos en `feature/pruebas-dependencias-<apellido>`; por ejemplo: `test(dependencias): agregar matriz de pruebas de productos`, `feat(categorias): navegar a los productos de una categoría`, `docs: agregar informe de hallazgos`.

- Guarda en el repositorio `docs/matriz-pruebas-s8.xlsx` (o `.md`), `docs/informe-hallazgos-s8.pdf` y la colección exportada `postman/PharmaSoft-S8-Dependencias.json`.
- Abre una solicitud de incorporación hacia `develop` y sube el informe al aula virtual como `Apellido_LP2_S8_Autonoma.pdf`.

Producto esperado

- Matriz con al menos 18 casos ejecutados (14 base y 4 propios), con resultado obtenido, estado y evidencia de cada uno.
- Informe de hallazgos en PDF con las secciones B1 a B5.
- Botón «Ver productos» funcionando entre Categorías y Productos.
- Colección de Postman exportada y rama con al menos tres commits y una solicitud de incorporación hacia `develop`.

Lista de cotejo

N.º	Criterio observable	Pts.	Sí	No
1	La matriz tiene al menos 18 casos, con los 14 base y 4 propios distribuidos en altas, cambios y bajas.	2		
2	Cada caso indica precondición, datos y resultado esperado deducido de la regla de negocio.	2		
3	Cada caso registra el resultado obtenido (SPA y código HTTP) y su estado Pasa o Falla.	2		
4	Las operaciones que la SPA bloquea se prueban también contra la API con Postman.	1		
5	Cada caso tiene su evidencia nombrada con el ID.	1		
6	Cada hallazgo tiene su ficha completa, con causa probable y corrección propuesta en la capa correcta.	2.5		
7	La tabla de cobertura indica dónde se valida cada regla de dependencia.	1		
8	La recomendación sobre el filtro describe el cambio en el endpoint, el repositorio y el servicio del frontend.	1		
9	Las cinco preguntas de análisis se responden con argumentos y referencias al código.	2.5		
10	«Ver productos» navega con `queryParams` y el listado llega filtrado por esa categoría.	3		
11	La rama tiene al menos tres commits en días distintos, la colección de Postman y los documentos en `docs/`.	1		
12	Existe la solicitud de incorporación hacia `develop` y el informe está en el aula virtual.	1		
	Total	**20**		

3. Criterios de entrega

- Plazo: **7 de octubre de 2026, 23:59**. En la sesión 9 (8 de octubre) trabajarás el formulario cabecera–detalle de Ventas, que depende a la vez de Clientes y Productos: tus hallazgos de hoy te servirán ahí.
- Las entregas tardías se califican sobre el 70 % del puntaje, hasta 24 horas después del plazo.
- Una matriz cuyos resultados no coincidan con las evidencias, o un repositorio sin commits propios, se califica con cero.
- Puedes usar asistentes de inteligencia artificial para consultar dudas, pero la matriz debe reflejar pruebas que ejecutaste tú; el docente podrá pedirte que repitas cualquier caso en clase.

4. Dudas frecuentes

Pregunta	Respuesta
El caso A-04 por Postman devolvió 201. ¿Me equivoqué?	No. Registra el resultado tal como ocurrió, marca el caso como **Falla** y documéntalo como hallazgo: la regla de negocio dice que un producto no debe quedar en una categoría inactiva.
¿Cómo sé el id de la categoría inactiva?	Ejecuta `GET {{baseUrl}}/categorias` en Postman y busca la que tiene `"estado": false`.
En C-03 no sé cuál es el resultado esperado.	Es intencional: la regla no está definida. Elige una, escríbela como resultado esperado y defiéndela en la pregunta 2 de B5.
¿Tengo que corregir el backend?	No. Propón la corrección en la ficha del hallazgo, con el archivo y el método que cambiarías. Si además la implementas en una rama propia del backend, menciónalo en el informe.
«Ver productos» abre /productos pero no filtra.	Revisa que la ruta use `[queryParams]` y que en `ProductoList` exista `readonly categoriaId = input<string>()`; el nombre del input debe ser idéntico al del parámetro.
¿Cómo exporto la colección de Postman?	Clic derecho sobre la colección → Export → Collection v2.1.

Cuando termines, habrás respondido la pregunta con la que empezó la sesión. En el Reto 01 hiciste que la interfaz guiara al jefe de almacén: solo ve categorías válidas, no puede dejar un producto sin clasificar y entiende por qué una categoría con productos no se elimina. En el Reto 02 comprobaste qué parte de esa protección vive realmente en el backend y qué parte depende de que el usuario use la SPA, y dejaste documentado cómo cerrar esas brechas. Esa combinación, una interfaz que orienta y una API que no se deja engañar, es la que necesitarás en la sesión 9, cuando una venta dependa a la vez de un cliente y de varios productos.

5. Referencias

Angular. (s. f.-a). *Common routing tasks*. https://angular.dev/guide/routing/common-router-tasks

Angular. (s. f.-b). *Reactive forms*. https://angular.dev/guide/forms/reactive-forms

International Software Testing Qualifications Board. (2023). *Certified Tester Foundation Level syllabus* (versión 4.0). ISTQB. https://www.istqb.org

Postman. (s. f.). *Send API requests and get response data in Postman*. https://learning.postman.com/docs/sending-requests/requests/

Spring. (s. f.). *Spring Data JPA: Query methods*. https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html